

- Dockerfile 및 실행 파일 세팅

mkdir 명령어로 실습을 위한 hello-ssh 폴더를 만들고, hello-ssh 폴더 안에 nano 명령어를 이용해 간단한 문장을 출력하는 hello.c 파일을 만들고 Dockerfile을 작성했다. Dockerfile의 내용은 아래와 같다.

```
FROM ubuntu:22.04
WORKDIR /app
COPY hello.c .
RUN apt-get update && apt-get install -y build-essential
RUN gcc -o hello hello.c
EXPOSE 8080
CMD ["/hello"]
```

- 이미지 빌드 및 확인

sudo docker build -t <이미지명> <경로>로 이미지를 빌드하였다. 그 후 sudo docker image ls 명령어를 이용하여 목록을 보면 hello-ssh로 빌드된 것을 확인할 수 있다.

```
user@DESKTOP-CS2C1DH:~/hello-ssh$ sudo docker build -t hello-ssh .
[+] Building 37.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 195B
=> [internal] load metadata for docker.io/library/ubuntu:24.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/ubuntu:24.04@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
=> => resolve docker.io/library/ubuntu:24.04@sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
=> [internal] load build context
=> => transferring context: 28B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY hello.c .
=> [4/5] RUN apt-get update && apt-get install -y build-essential
=> [5/5] RUN gcc -o hello hello.c
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:84b8fa977c552aa30b4d0752ccc81ee7274a2641f86b5c1e4df679c99ff970ee
=> => exporting config sha256:2d853bc0807e49f90e656c16af59db53c39b7c4157ccf8a3bdc0364c01683019
=> => exporting attestation manifest sha256:88f84ceee25f4da03598e51ec138c576567b78b85ec6a98cb9ab4742f73f608
=> => exporting manifest list sha256:2ea10fc3aac7f7a335803ad6eab58a1bad3f2255e074e9abbf2f7fdd3c4de9da
=> => naming to docker.io/library/hello-ssh:latest
=> => unpacking to docker.io/library/hello-ssh:latest
user@DESKTOP-CS2C1DH:~/hello-ssh$ sudo docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-ssh:latest	2ea10fc3aac7	677MB	188MB	

- 컨테이너 실행 및 확인

일반적인 sudo docker run 명령어로 컨테이너를 실행하니 컨테이너가 바로 종료되는 문제가 발생했다. 그래서 뒤에 tail -f /dev/null를 붙여 종료되지 않도록 했다. 컨테이너가 실행중임을 확인하기 위해 sudo docker ps 명령어를 사용하니 CONTAINER ID 등의 정보를 확인할 수 있다.

```
user@DESKTOP-CS2C1DH:~/hello-ssh$ sudo docker run --pid=host -d -p 7777:8080 hello-ssh tail -f /dev/null
7736eb2e54da77995235ce9f549012b12cc5c99d2104b7bd28050f3720ff7dcb
user@DESKTOP-CS2C1DH:~/hello-ssh$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
7736eb2e54da	hello-ssh	"tail -f /dev/null"	59 seconds ago	Up 58 seconds	0.0.0.0:7777->8080/tcp, [::]:7777->8080/tcp	confident_proskuriakova

- 컨테이너 쉘 접속 및 파일 시스템 탐색

sudo docker ps 명령어로 확인한 CONTAINER ID을 넣어 쉘에 접속했다.

```
user@DESKTOP-CS2C1DH:~/hello-ssh$ sudo docker exec -it 7736eb2e54da /bin/sh
# pwd
/app
# ls
hello hello.c
# ls /
app bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
# cd /lib
# ls
apt          compat-ld   dpkg        gnupg       gold-ld     locale      mime         sasl2       systemd     udev
bfd-plugins  cpp         gcc         gnupg2      init        lsb         os-release  sysctl.d    tmpfiles.d  x86_64-linux-gnu
# ldd /app/hello
linux-ld.so.1 (0x00007fffd8517d000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ff197ebf000)
/lib64/ld-linux-x86-64.so.2 (0x00007ff1980db000)
# exit
user@DESKTOP-CS2C1DH:~/hello-ssh$ |
```

ldd /app/hello 명령어로 libc 경로까지 확인했다.